



ΣΧΕΔΙΑΣΜΟΣ ΟΛΟΚΛΗΡΩΜΕΝΩΝ ΚΥΚΛΩΜΑΤΩΝ II

# 8-bit Radix-4 Booth Multiplier

---

## Ομάδα 7

Βασίλειος Βλάχος (ΑΜ: 1100832)

Βιολέτα Μέγαρη (ΑΜ: 1100938)

# Γιατί Radix-4 Booth;



## Radix-2

Ένας τυπικός πολλαπλασιαστής απαιτεί δημιουργία και άθροιση 8 μερικών γινομένων για δύο όρους των 8 bits.



## Radix-4

Η κωδικοποίηση Booth μειώνει τα μερικά γινόμενα από 8 σε μόλις 4, μέσω ολισθήσεων κατά 2 bits.



## Το πρόβλημα

Χρειάζονται «hard multiples» (π.χ.  $3Y$ ). Αυτά υπολογίζονται ως  $4Y - Y$ :  
- $Y$  στο τρέχον μερικό γινόμενο και  
+ $Y$  στο επόμενο, ώστε να αποφευχθεί αργός CPA.

**Βασική ιδέα:** χρήση ολισθήσεων και αφαιρέσεων αντί πολλαπλασιασμών, ώστε να παραχθούν όλα τα πολλαπλάσια του όρου  $Y$ .

# Radix-4 Modified Booth Encoding

$P = Y \times X$ , με  $X = x_7x_6...x_1x_0$ . Προσθέτουμε bit  $x_{-1} = 0$  και ομαδοποιούμε σε τριάδες:

$x_1x_0x_{-1} \rightarrow x_3x_2x_1 \rightarrow x_5x_4x_3 \rightarrow x_7x_6x_5$

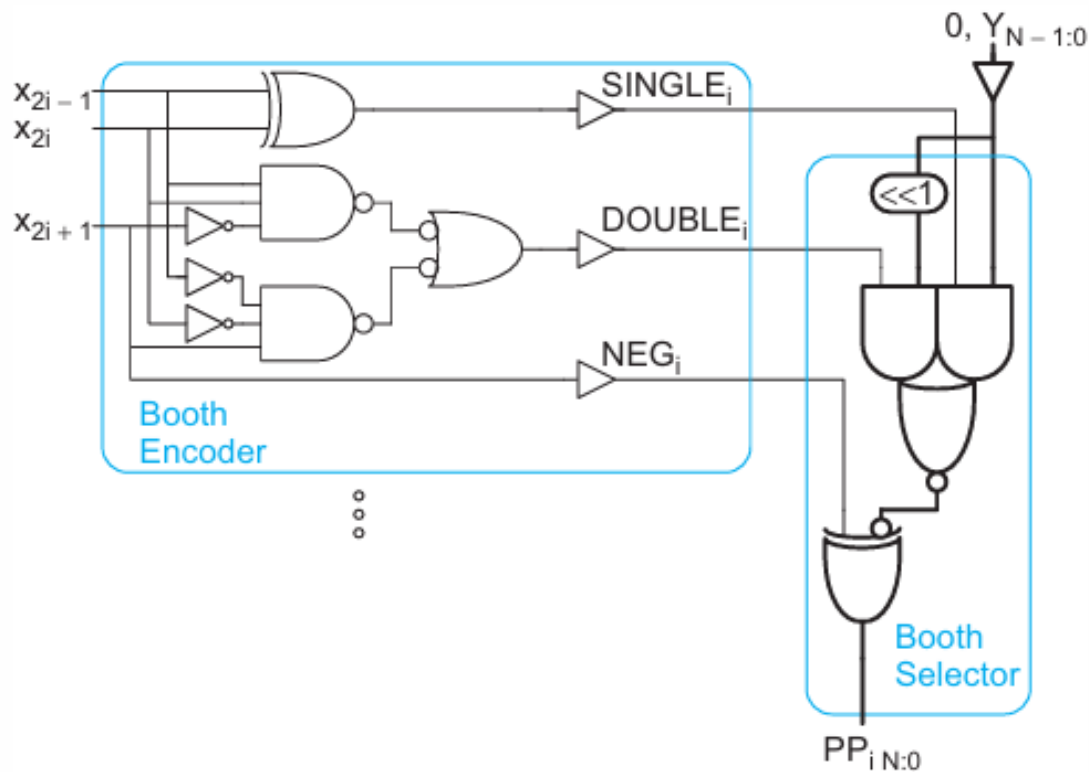
**Κάθε τριάδα παράγει 3 σήματα ελέγχου:**

- SINGLE — επιλογή  $\pm 1 \times Y$
- DOUBLE — επιλογή  $\pm 2 \times Y$
- NEG — αφαίρεση ( $Y$  σε συμπλήρωμα)

| Inputs     |          |            | Partial Product | Booth Selects              |                            |                         |
|------------|----------|------------|-----------------|----------------------------|----------------------------|-------------------------|
| $x_{2i+1}$ | $x_{2i}$ | $x_{2i-1}$ | $PP_i$          | SINGLE <sub><i>i</i></sub> | DOUBLE <sub><i>i</i></sub> | NEG <sub><i>i</i></sub> |
| 0          | 0        | 0          | 0               | 0                          | 0                          | 0                       |
| 0          | 0        | 1          | $Y$             | 1                          | 0                          | 0                       |
| 0          | 1        | 0          | $Y$             | 1                          | 0                          | 0                       |
| 0          | 1        | 1          | $2Y$            | 0                          | 1                          | 0                       |
| 1          | 0        | 0          | $-2Y$           | 0                          | 1                          | 1                       |
| 1          | 0        | 1          | $-Y$            | 1                          | 0                          | 1                       |
| 1          | 1        | 0          | $-Y$            | 1                          | 0                          | 1                       |
| 1          | 1        | 1          | $-0 (= 0)$      | 0                          | 0                          | 1                       |

Πίνακας 11.12 — N. Weste & D. Harris, CMOS VLSI Design

# Booth Encoder & Selector



## Booth Encoder

Παίρνει την τριάδα ( $x_{2i+1}$ ,  $x_{2i}$ ,  $x_{2i-1}$ ) και παράγει τα  $SINGLE_i$ ,  $DOUBLE_i$ ,  $NEG_i$  με απλές πύλες (XOR, NAND).



## Booth Selector

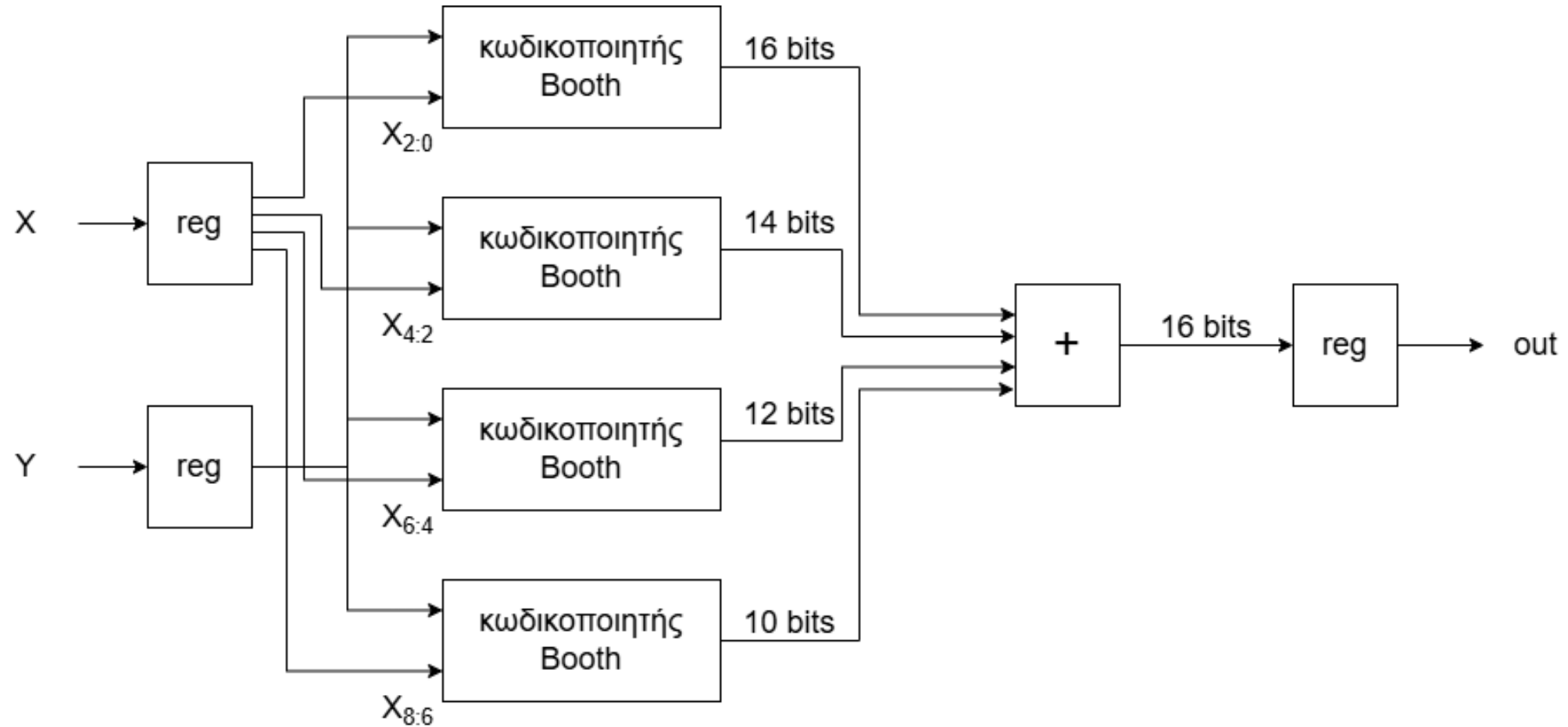
Χρησιμοποιεί τα select bits για να επιλέξει 0, Y ή 2Y (ολίσθηση  $\ll 1$ ) και έπειτα το συμπληρώνει αν  $NEG_i = 1$ .



## 1-bit δομικό στοιχείο

Και τα δύο κυκλώματα σχεδιάστηκαν ως 1-bit modules και επαναλαμβάνονται (generate) για όλα τα bits του Y.

# Partial Product Generation



Τα  $X$  και  $Y$  καταχωρούνται, το  $X$  διασπάται σε 4 τριάδες ( $x_{2:0}$ ,  $x_{4:2}$ ,  $x_{6:4}$ ,  $x_{8:6}$ ) και κάθε κωδικοποιητής Booth παράγει ένα sign-extended μερικό γινόμενο πλάτους 16, 14, 12 και 10 bits αντίστοιχα.

# Πρόσθεση: Carry-Select Δέντρο

$$\begin{array}{r} x_7 \ x_6 \ x_5 \ x_4 \ x_3 \ x_2 \ x_1 \ x_0 \\ \times \ y_7 \ y_6 \ y_5 \ y_4 \ y_3 \ y_2 \ y_1 \ y_0 \\ \hline \end{array}$$

x7 x6 x5 x4 x3 x2 x1 x0

**×** y7 y6 y5 y4 y3 y2 y1 y0 0

---

x7 x6 x5 x4 x3 x2 x1 x0

×

y7 y6 y5 y4 y3 y2 y1 y0 0

---



x7 x6 x5 x4 x3 x2 x1 x0



y7 y6 y5 y4 y3 y2 y1 y0 0

sa sa sa sa sa sa sa sa a7 a6 a5 a4 a3 a2 a1 a0

sb sb sb sb sb sb b7 b6 b5 b4 b3 b2 b1 b0

sc sc sc sc c7 c6 c5 c4 c3 c2 c1 c0

sd sd d7 d6 d5 d4 d3 d2 d1 d0

x7 x6 x5 x4 x3 x2 x1 x0

✖

y7 y6 y5 y4 y3 y2 y1 y0 0

16 bit —→ sa sa sa sa sa sa sa sa a7 a6 a5 a4 a3 a2 a1 a0

14 bit —→ sb sb sb sb sb sb b7 b6 b5 b4 b3 b2 b1 b0

12 bit —→ sc sc sc sc c7 c6 c5 c4 c3 c2 c1 c0

10 bit —→sd sd d7 d6 d5 d4 d3 d2 d1 d0

x7 x6 x5 x4 x3 x2 x1 x0

✖

y7 y6 y5 y4 y3 y2 y1 y0 0

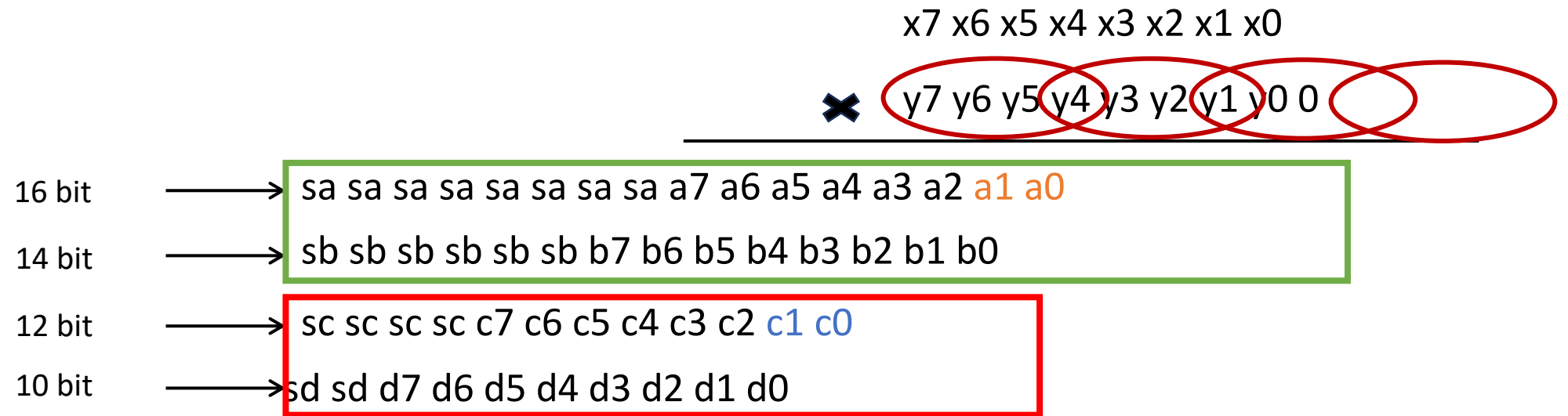
16 bit —→ sa sa sa sa sa sa sa sa a7 a6 a5 a4 a3 a2 a1 a0

14 bit —→ sb sb sb sb sb sb b7 b6 b5 b4 b3 b2 b1 b0

12 bit —→ sc sc sc sc c7 c6 c5 c4 c3 c2 c1 c0

10 bit —→sd sd d7 d6 d5 d4 d3 d2 d1 d0

Κρατάμε αυτά τα  
a1 a0 και c1 c0



csa 14 bit

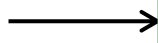


x7 x6 x5 x4 x3 x2 x1 x0



y7 y6 y5 y4 y3 y2 y1 y0 0

16 bit



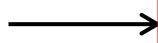
sa sa sa sa sa sa sa sa a7 a6 a5 a4 a3 a2 a1 a0

14 bit



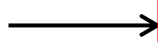
sb sb sb sb sb sb b7 b6 b5 b4 b3 b2 b1 b0

12 bit



sc sc sc sc c7 c6 c5 c4 c3 c2 c1 c0

10 bit



sd sd d7 d6 d5 d4 d3 d2 d1 d0

csa 10 bit



e13 e12 e11 e10 e9 e8 e7 e6 e5 e4 e3 e2 e1 e0

f9 f8 f7 f6 f5 f4 f3 f2 f1 f0

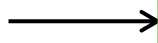
csa 14 bit

x7 x6 x5 x4 x3 x2 x1 x0

✕

y7 y6 y5 y4 y3 y2 y1 y0 0

16 bit



sa sa sa sa sa sa sa sa a7 a6 a5 a4 a3 a2 a1 a0

14 bit



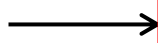
sb sb sb sb sb sb b7 b6 b5 b4 b3 b2 b1 b0

12 bit



sc sc sc sc c7 c6 c5 c4 c3 c2 c1 c0

10 bit

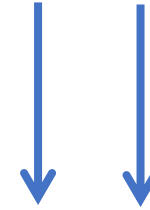


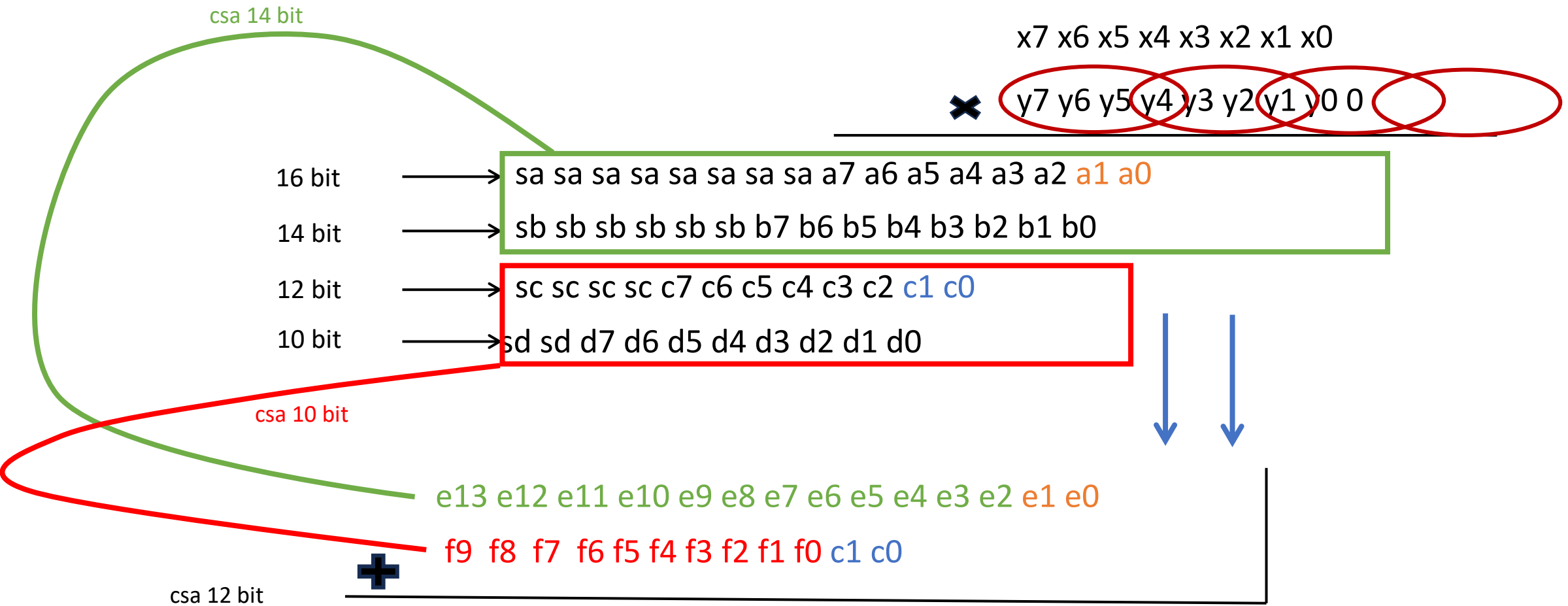
sd sd d7 d6 d5 d4 d3 d2 d1 d0

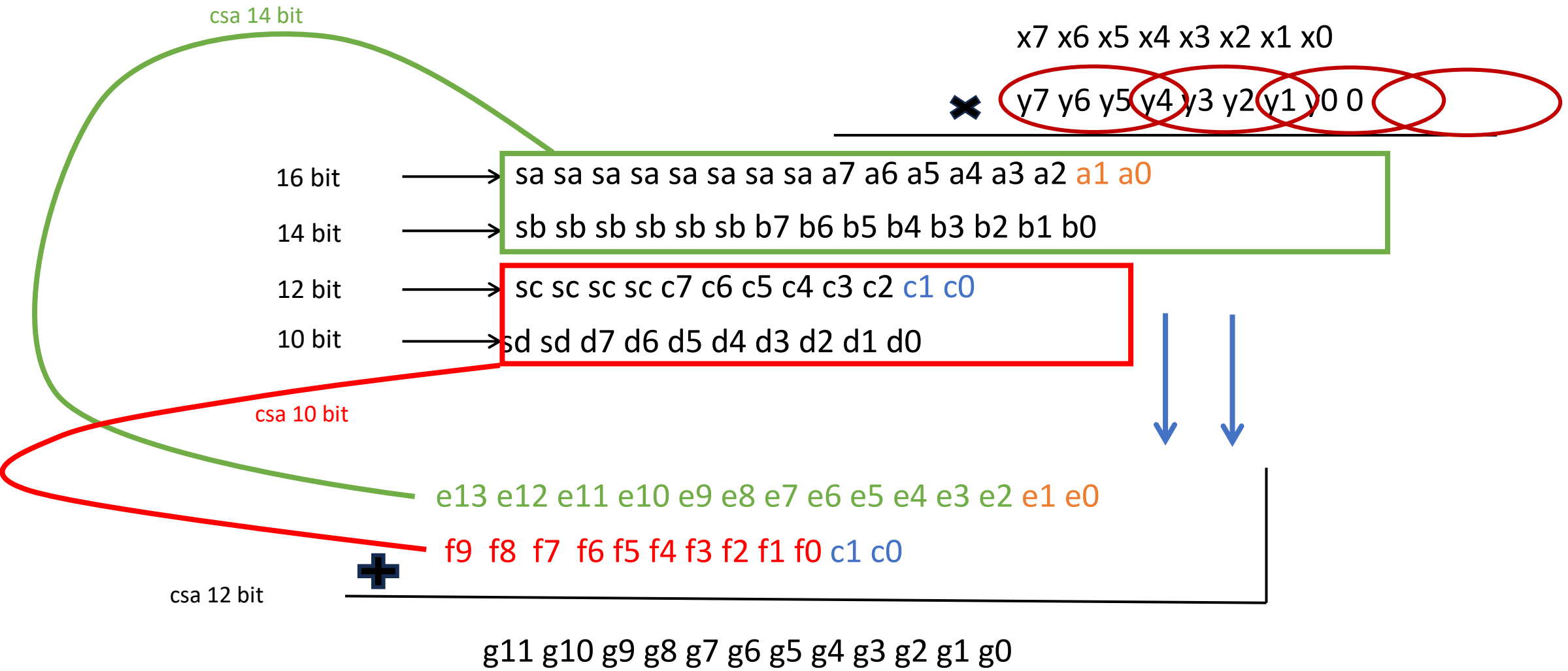
csa 10 bit

e13 e12 e11 e10 e9 e8 e7 e6 e5 e4 e3 e2 e1 e0

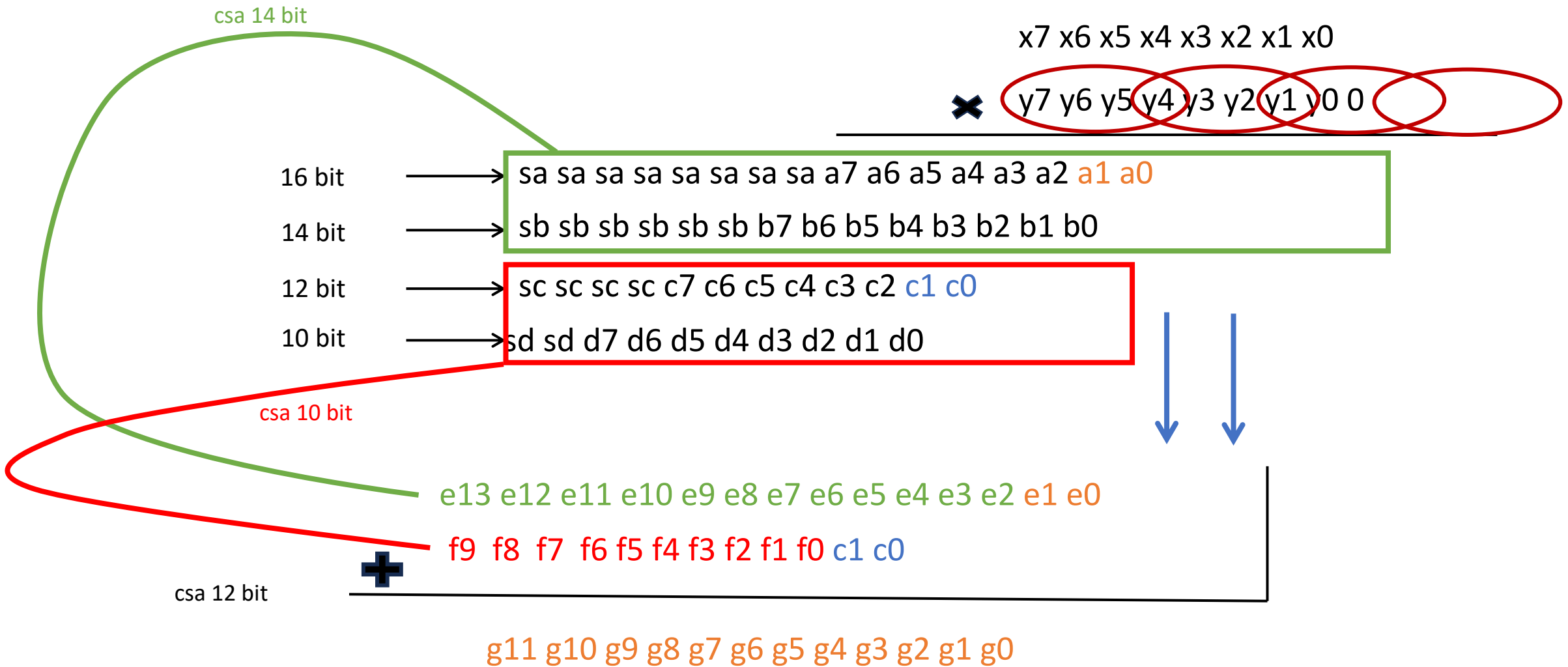
f9 f8 f7 f6 f5 f4 f3 f2 f1 f0 c1 c0











Τελικό Αποτέλεσμα :

csa 14 bit

x7 x6 x5 x4 x3 x2 x1 x0

×

y7 y6 y5 y4 y3 y2 y1 y0 0

16 bit

sa sa sa sa sa sa sa sa a7 a6 a5 a4 a3 a2 a1 a0

14 bit

sb sb sb sb sb sb b7 b6 b5 b4 b3 b2 b1 b0

12 bit

sc sc sc sc c7 c6 c5 c4 c3 c2 c1 c0

10 bit

sd sd d7 d6 d5 d4 d3 d2 d1 d0

csa 10 bit

e13 e12 e11 e10 e9 e8 e7 e6 e5 e4 e3 e2 e1 e0

f9 f8 f7 f6 f5 f4 f3 f2 f1 f0 c1 c0

csa 12 bit

+

g11 g10 g9 g8 g7 g6 g5 g4 g3 g2 g1 g0

Τελικό Αποτέλεσμα :

csa 14 bit

x7 x6 x5 x4 x3 x2 x1 x0

×

y7 y6 y5 y4 y3 y2 y1 y0 0

16 bit

→

sa sa sa sa sa sa sa sa a7 a6 a5 a4 a3 a2 a1 a0

14 bit

→

sb sb sb sb sb sb b7 b6 b5 b4 b3 b2 b1 b0

12 bit

→

sc sc sc sc c7 c6 c5 c4 c3 c2 c1 c0

10 bit

→

sd sd d7 d6 d5 d4 d3 d2 d1 d0

csa 10 bit

e13 e12 e11 e10 e9 e8 e7 e6 e5 e4 e3 e2 e1 e0

f9 f8 f7 f6 f5 f4 f3 f2 f1 f0 c1 c0

+

csa 12 bit

g11 g10 g9 g8 g7 g6 g5 g4 g3 g2 g1 g0

Τελικό Αποτέλεσμα : g11 g10 g9 g8 g7 g6 g5 g4 g3 g2 g1 g0 e1 e0 a1 a0

csa 14 bit

x7 x6 x5 x4 x3 x2 x1 x0

×

y7 y6 y5 y4 y3 y2 y1 y0 0

16 bit

→

sa sa sa sa sa sa sa sa a7 a6 a5 a4 a3 a2 a1 a0

14 bit

→

sb sb sb sb sb sb b7 b6 b5 b4 b3 b2 b1 b0

12 bit

→

sc sc sc sc c7 c6 c5 c4 c3 c2 c1 c0

10 bit

→

sd sd d7 d6 d5 d4 d3 d2 d1 d0

csa 10 bit

e13 e12 e11 e10 e9 e8 e7 e6 e5 e4 e3 e2 e1 e0

f9 f8 f7 f6 f5 f4 f3 f2 f1 f0 c1 c0

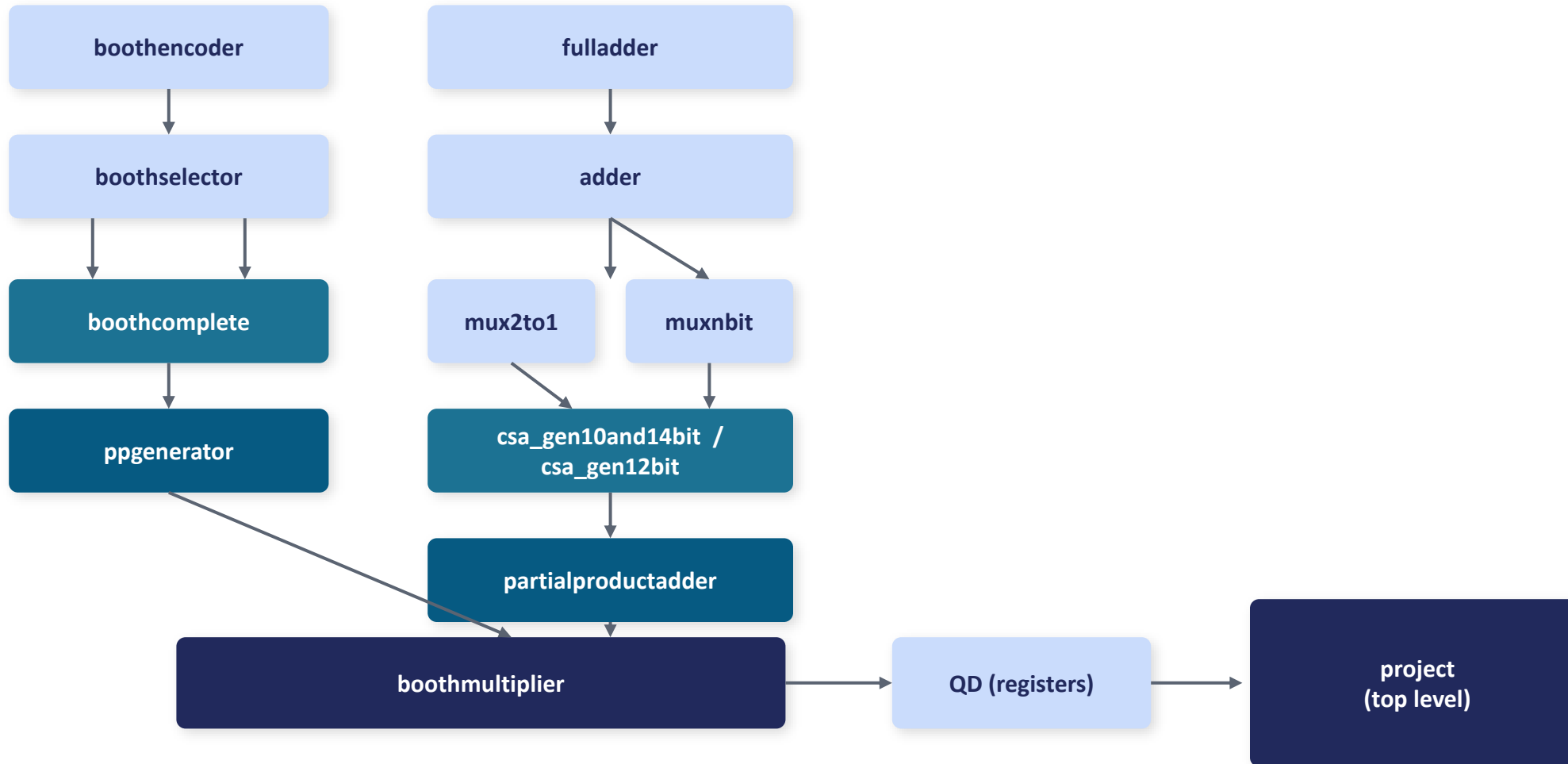
+

csa 12 bit

g11 g10 g9 g8 g7 g6 g5 g4 g3 g2 g1 g0

Τελικό Αποτέλεσμα : g11 g10 g9 g8 g7 g6 g5 g4 g3 g2 g1 g0 e1 e0 a1 a0

# Ιεραρχία Δομικών Στοιχείων



VHDL δομικού τύπου (structural): κάθε επίπεδο χτίζεται αποκλειστικά μέσω instantiation προηγούμενων components.

# 45 nm vs 7 nm

| Μετρική           | 45 nm                   | 7 nm                   |
|-------------------|-------------------------|------------------------|
| Περίοδος ρολογιού | 5.000 ns                | 0.705 ns               |
| Critical path     | 4.861 ns                | 0.684 ns               |
| Slack             | 13 ps                   | 2 ps                   |
| Πλήθος κελιών     | 208                     | 405                    |
| Συνολικό εμβαδόν  | 946.951 $\mu\text{m}^2$ | 42.049 $\mu\text{m}^2$ |
| Κατανάλωση ισχύος | 207.79 $\mu\text{W}$    | 211.14 $\mu\text{W}$   |

7x

ταχύτερο critical path στα 7 nm (0.684 ns έναντι 4.861 ns)

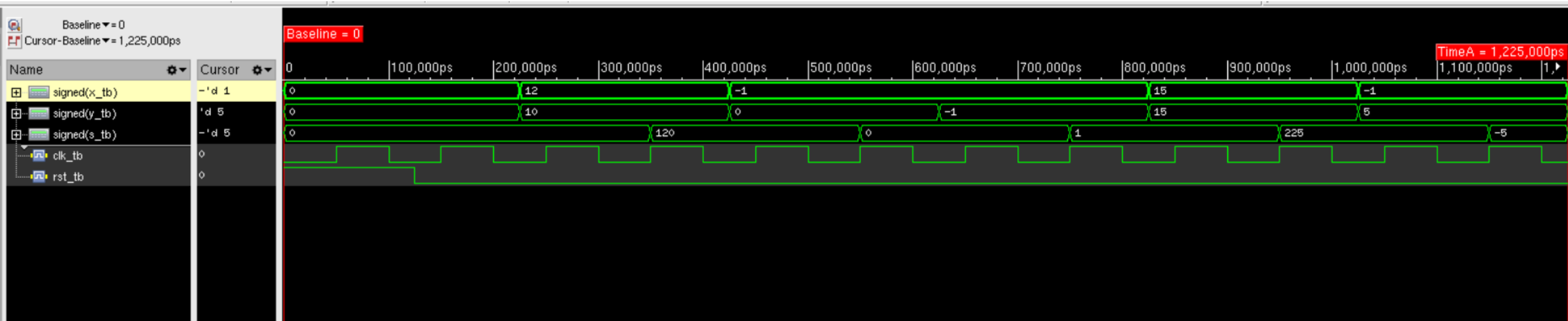
22x

μικρότερο εμβαδόν στα 7 nm, παρά το μεγαλύτερο πλήθος κελιών

≈ ίδια ισχύς

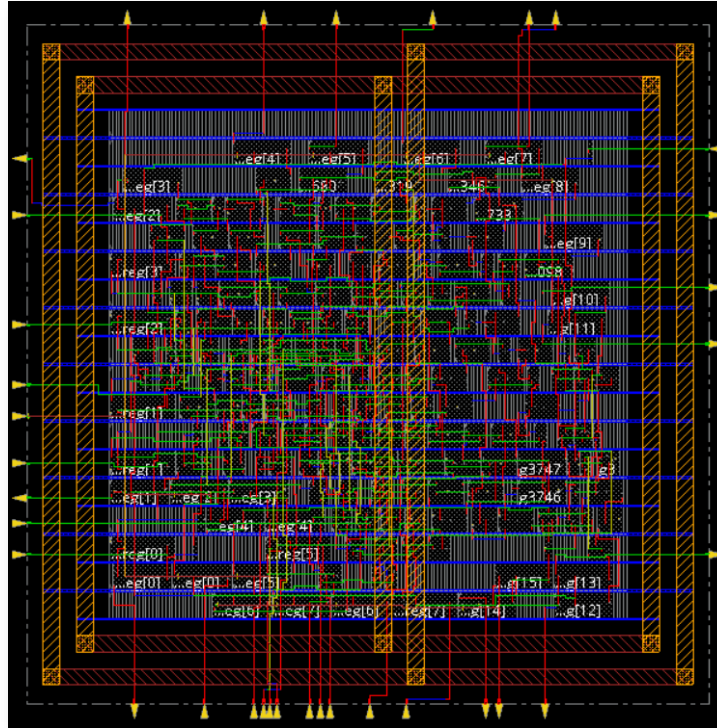
207.8  $\mu\text{W}$  → 211.1  $\mu\text{W}$ : το μικρότερο μέγεθος αντισταθμίζεται από υψηλότερη πυκνότητα διαρροής στα 7 nm

# XCELIUM & LEC



- Ο σχεδιασμός πέρασε το LEC Validation Test
- Ενδεικτικές τιμές για επαλήθευση μέσω Testbench στο Xcelium

# Layout (Innovus, 45 nm)



- Τα I/O pads τοποθετήθηκαν με βάση το optimal placement του εργαλείου.
- Έχει εφαρμοστεί clock tree synthesis (CTS) για ομοιόμορφη διανομή ρολογιού.
- Το floorplan βασίζεται στα αποτελέσματα σύνθεσης Genus στα 45 nm.



# Ανακεφαλαίωση

- 1 Υλοποιήσαμε πλήρη 8-bit radix-4 Booth multiplier σε δομική VHDL, από τα gate-level modules μέχρι το top-level σύστημα με καταχωρητές.
- 2 Η κωδικοποίηση Booth μείωσε τα μερικά γινόμενα από 8 σε 4, ενώ η χρήση Carry-Select αθροιστών μεγιστοποίησε την ταχύτητα πρόσθεσής τους.
- 3 Η σύνθεση σε δύο τεχνολογίες (45 nm, 7 nm) επιβεβαίωσε τα αναμενόμενα οφέλη scaling: μικρότερο critical path, λιγότερο εμβαδόν, με μικρή αύξηση ισχύος.
- 4 Η ροή ολοκληρώθηκε έως το physical layout, με χρήση Genus για σύνθεση και Innovus για place & route.



ΣΧΕΔΙΑΣΜΟΣ ΟΛΟΚΛΗΡΩΜΕΝΩΝ ΚΥΚΛΩΜΑΤΩΝ II

# Ευχαριστούμε

*Ερωτήσεις;*

---

Βλάχος Βασίλειος  
Μέγαρη Βιολέτα

